

Concurrency & Distributed Systems

Week 3: Synchronization, Memory Semantics & Deadlocks

Day 1 — Part 1: Race Conditions, Monitors, and Lock Identity

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

Week 3 — Execution Control

Theme: From uncontrolled interleaving to controlled coordination

Today:

- What exactly is a race condition?
- What does `synchronized` really do?
- What is a monitor?
- Why does lock identity matter?

Goal

Move from accidental execution to disciplined coordination over shared state.

Race Review — Broken Counter

```
static int count = 0;

static void increment() {
    count++;
}
```

Prediction:

If we run:

- 4 threads
- each increments 1000 times

Question: What value should count have if the program is correct?

Invariant Thinking

Expected Invariant

$$count = threads \times iterations$$

If that invariant fails, correctness has failed.

Core question

Why does it fail?

What Is a Race Condition?

Operational Definition

A race condition occurs when:

- two or more threads access the same variable,
- at least one access is a write,
- and no coordination enforces safe ordering.

Practical Rule

Race condition = shared mutable state + concurrent access + at least one write + no synchronization

Why `count++` Is Not Safe

```
count++;
```

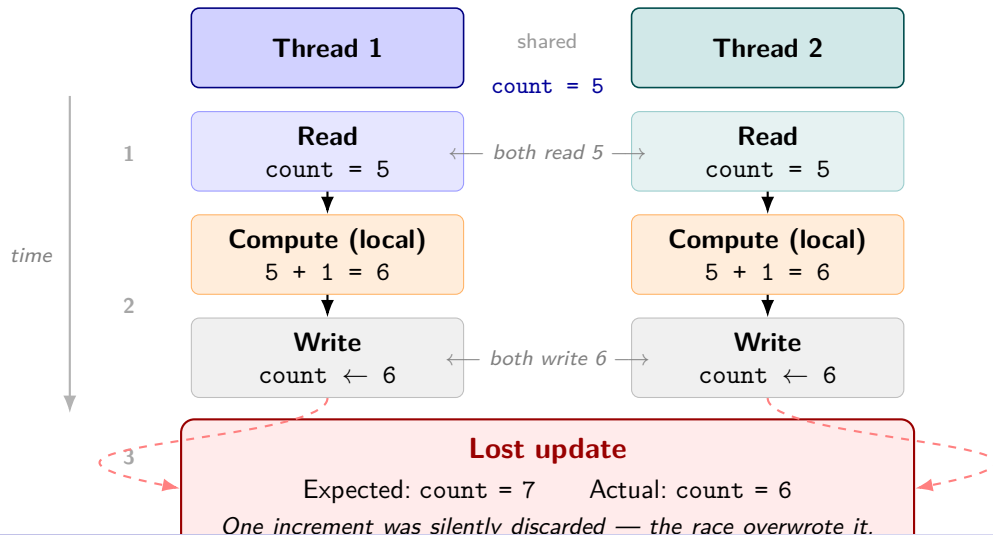
This looks like one operation in source code, but it is really:

read *count* → add 1 → write back

Problem

If two threads interleave these steps, one update can overwrite another.

Interleaving and Lost Update



Quick Checkpoint

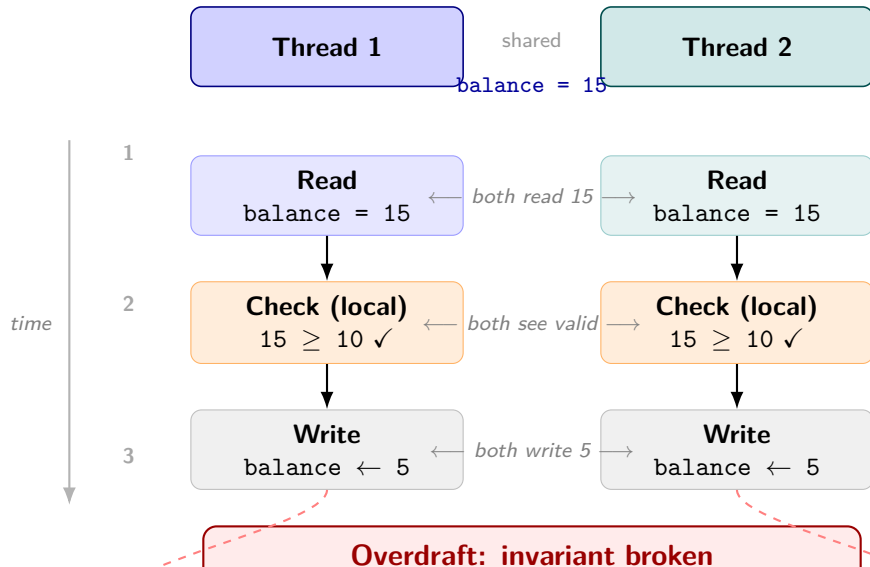
Think-Pair-Share

If two threads withdraw \$10 from a bank account with a \$15 balance, and there is no synchronization, what final balances are possible?

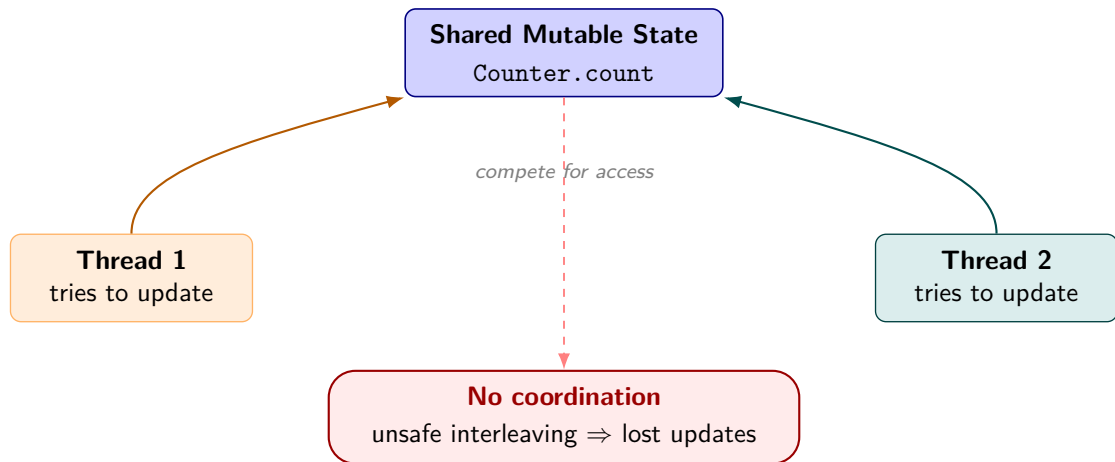
Think about:

- What each thread may read
- Whether both threads may believe the withdrawal is valid
- Which invariant is broken

Bank Account Race — One Unsafe Interleaving



Mental Model of the Problem



Observation

The problem is not randomness in Java

From Chaos to Control

Problem

Multiple threads access the same mutable state without coordination.

Question

How do we prevent this chaos?

We need **mutual exclusion**.

A Simple Analogy for a Lock

Analogy

A lock is like a room with a single key.

- If one person holds the key, only that person may enter
- Everyone else must wait outside
- When the key is returned, another person may enter

Mapping

Thread = person Lock = key Critical section = room

Introducing synchronized

```
synchronized(lockObject) {  
    // critical section  
}
```

Behavioral Guarantee

Only one thread executes inside the critical section at a time, provided all threads synchronize on the same lock object.

This is **mutual exclusion**.

synchronized Method vs Block

```
public synchronized void increment() {  
    count++;  
}
```

```
public void increment() {  
    synchronized(this) {  
        count++;  
    }  
}
```

Key idea

For an instance method, `synchronized` uses the monitor of `this`.

Important

A `synchronized` block gives more control because you choose the lock object explicitly.

What Is a Monitor?

Main Idea

In Java, every object has an intrinsic monitor.

- A thread can acquire the monitor
- Other threads trying to enter become blocked
- The monitor is released when the synchronized region exits

Key Point

Synchronization works through object identity.

Concept Check — BLOCKED vs WAITING

BLOCKED

A thread is **BLOCKED** when it is trying to enter a synchronized region but another thread already holds the monitor.

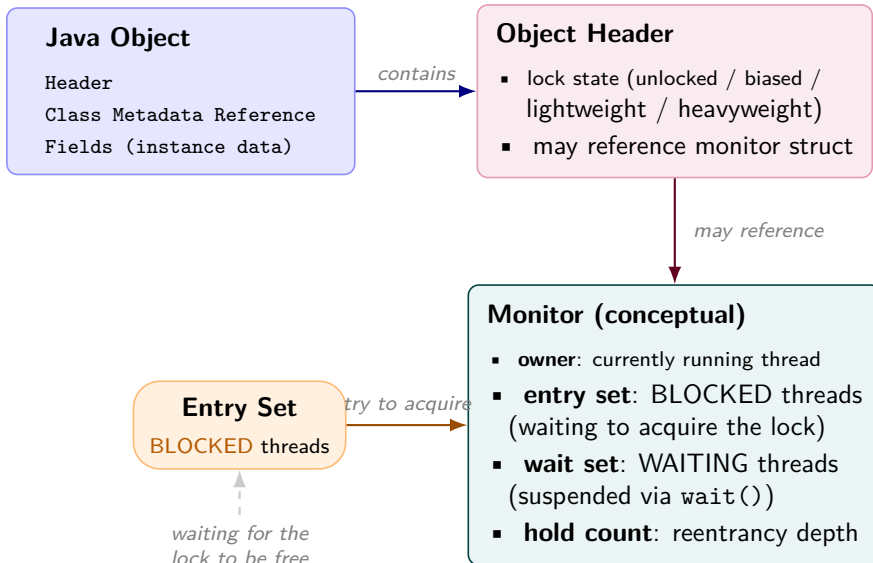
WAITING

A thread is **WAITING** when it has voluntarily paused, typically as part of condition coordination such as `wait()`.

For today

We focus on monitor contention and BLOCKED threads. Condition waiting will be treated later.

Monitor Layout (Conceptual View)



Reentrancy in One Sentence

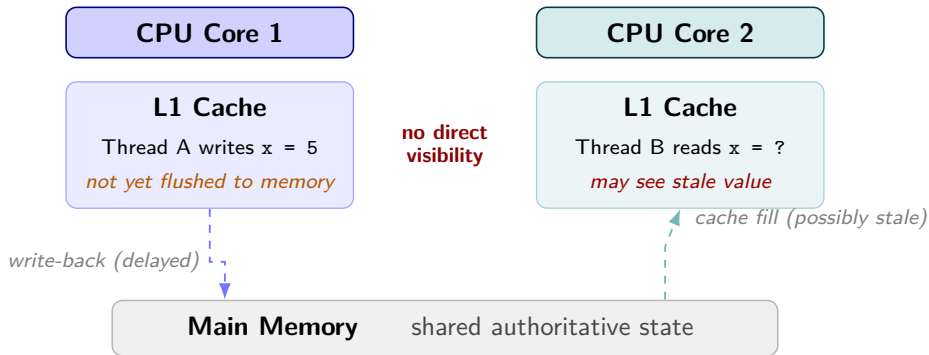
Reentrancy

If a thread already holds a monitor, it may enter another synchronized region guarded by the same object without blocking itself.

Why it matters

Without reentrancy, many ordinary object-oriented method calls would deadlock immediately.

Visibility Intuition — Caches and Main Memory



Why synchronization matters for visibility

- Without synchronization, Thread B may read a value written before Thread A's update became visible.
- synchronized flushes the writer's cache and invalidates stale entries on the reader's side.
- This is formalized as a **happens-before** relation in the Java Memory Model.

Connecting Back to the Race

The counter failed because:

- operations interleaved without mutual exclusion
- no synchronization guaranteed visibility between threads

Important

Monitors are not only about blocking overlap. They also support memory visibility guarantees in the Java Memory Model.

Bridge to next part

In the Java Memory Model, synchronization establishes a **happens-before** relationship between threads.

Protecting the Counter with a Shared Lock

```
class Counter {  
    static int count = 0;  
    static final Object lock = new Object();  
  
    static void increment() {  
        synchronized(lock) {  
            count++;  
        }  
    }  
}
```

Interpretation

The increment is now protected by one shared monitor. Only one thread may execute that critical section at a time.

Lock Identity Trap

```
public void withdraw(int amount) {  
    Object lock = new Object();  
    synchronized(lock) {  
        balance -= amount;  
    }  
}
```

Question

Why does this fail to protect balance?

The Lock Identity Rule

Each call creates a new lock object.

Different threads therefore lock different objects.

Critical Rule

Synchronization works only when all threads lock the **same identity**.

Spot the Bug

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        synchronized(new Object()) {  
            count++;  
        }  
    }  
}
```

Answer

This creates a fresh lock object every time. No shared monitor means no real coordination.

A Common Misconception

Important

Synchronizing one method does not automatically make all related code thread-safe.

- All access paths to shared mutable state matter
- Reads may also require synchronization
- Thread safety is a property of the design, not one keyword

Reflection

How does the distinction between:

- BLOCKED
- WAITING

relate to:

- protecting a shared monitor identity
- waiting for a condition via `wait()` / `notify()`

Write one sentence connecting the concepts.

Takeaways

- Race conditions come from unsynchronized access to shared mutable state
- `count++` is not atomic
- `synchronized` provides mutual exclusion
- Every Java object has an intrinsic monitor
- Synchronization depends on **shared lock identity**
- Synchronization also supports visibility guarantees

Next

We now go deeper into what `synchronized` guarantees: happens-before, visibility, and memory semantics.